Project code: chat-rendering-troubleshooting

The code for this example project is in the ch08/chat-rendering-troubleshooting folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch08/chat-rendering-troubleshooting
```

You'll also need either the OpenAI API or the Google API keys to access their REST API. For details on creating API keys, refer to the appendix:

```
// .env
OPENAI_API_KEY=<OPEN_AI_SECRET_KEY>
GEMINI_API_KEY=<GOOGLE_AI_SECRET_KEY>
```

In production environments, you won't have access to the development error overlay, so it's best to use an *error boundary* component, as this will allow users to see meaningful error messages instead of blank screens and will present them with a list of recovery call-to-action buttons. Instructions on how to create an error boundary component are listed in detail in the official React reference docs (https://mng.bz/mZqW).

After handling basic application setup, you will often encounter challenges related to UI rendering, particularly when dealing with asynchronous responses from server handlers. These problems are especially prevalent in AI applications due to streaming responses and dynamic content updates. Let's look at the main categories of problems and their solutions.

Scroll behavior problems

Automatic scrolling happens when new messages arrive and get rendered on the screen and the app scrolls down to the last message. While I covered the basic implementation in chapter 2, several things can affect automatic scrolling behavior in AI chat applications.

Note that the placement of the scroll anchor element is crucial. Moving it outside the message list container or in a different order can break the scrolling behavior:

```
{
conversation.length > 0 && <ChatList
messages={conversation} isLoading={isLoading} />}
        <div ref={messageEndRef} />
```

Another problem you might find is that while the UI scrolls to the last message, it's still not scrolling down enough for the user to see the last sentence. This happens because there should be a gap at the bottom of the list to allow for the message box to appear. For instance, if a user sends a lengthy message that fills up the chat window, the last part of that message might be obscured by the input field at the bottom, making it